# Лабораторна робота 9. Навігація та збереження замовлень у Blazor

## Завдання та очікуваний результат

Додайте до каталогу й кошика сторінку перевірки замовлення, збереження його до SQLite, список замовлень і сторінку деталей за номером. Основне питання: як користувач переходить між сторінками і як номер у URL визначає дані компонента?

1. Маршрути `/checkout`, `/orders` і `/orders/{Id:int}`.
2. Єдина навігація в наявному макеті; підсвічування активної секції.
3. Збереження позицій і суми без форми покупця.
4. Розрізнення порожнього списку, відсутнього запису та помилки БД.
5. Кошик очищується тільки після успішного збереження.

Продовжуйте власний проєкт із лабораторної 8. Не створюйте новий шаблон і не замінюйте свою тему демонстраційною. Перед початком збережіть попередній результат комітом або окремою копією.

Готовий контрольний варіант — папка `Lab09_CampusCafe`. У ній застосунок `CampusCafe`, перевірки `CampusCafe.Checks` та README. Демо є результатом саме цього етапу, а не початковим шаблоном для механічної заміни власного проєкту.

**Межі навчального застосунку:** оплати, автентифікації та розмежування доступу немає. Список замовлень спільний, а не захищений список «моїх» покупок. Запускайте демо локально та вводьте тільки вигадані дані. Публікація вихідного коду не означає готовність сервісу приймати реальних покупців.

## Як працювати з кодом

У прикладі збережено назви `CampusCafe`, `Drink` і кавові опції. У своєму застосунку продовжуйте тему, обрану раніше: букети, фотодрук, подарункові набори, сендвічі чи інший каталог. Під час замін не втрачайте власні назви, формули та оформлення.

Повні лістинги показують узгоджений результат кроку. Перед додаванням коду прочитайте розбір: визначте, які дані файл отримує, що змінює і хто його використовує. Для невеликих правок використано `diff`: рядок із `-` вилучається, із `+` додається без знака `+`; рядки без цих знаків — контекст. Заголовки `---`, `+++` і `@@` не копіюйте до програми. Для великих змін наведено повні версії «Було / Стало», для CSS — лише новий файл.

Для кожного кроку виконайте три дії: спочатку передбачте результат, потім внесіть зміни під свою тематику, після цього перевірте поведінку. У коротких нотатках фіксуйте не перелік скопійованих файлів, а причинний зв'язок: «цей метод потрібний для..., без нього...». Готовий демо-проєкт використовуйте для порівняння після власної спроби.

Шляхи в лістингах відраховано від папки `CampusCafe`, де лежить `.csproj`. Це не вимога називати власний проєкт так само. Після зміни простору імен узгоджуйте його в усіх файлах через Rename редактора.

Після завершення кожного логічного етапу виконуйте `dotnet build`. Нові взаємозалежні класи спочатку додайте всі, а потім збирайте. Контрольні точки нижче вказують, коли очікується повністю працездатна версія.

## 1. Від кошика до збереженого замовлення

У попередній роботі `OrderState` жив у пам'яті інтерактивної сесії. Тепер користувач підтверджує замовлення, і сервіс створює постійний запис. Перезавантаження сторінки вже не повинно видаляти історію.

```
Home → кошик → /checkout
                   ↓ підтвердження
               PurchaseService → orders.db
                   ↓ номер нового запису
               /orders/12 ← посилання зі списку /orders
```

Маршрутизатор уже є в `Components/Routes.razor`; його не переносимо до `App.razor` і не додаємо `_Host.cshtml`. `@page` робить компонент сторінкою. `NavLink` — посилання з автоматичним класом `active`, а `NavigationManager.NavigateTo` виконує перехід із C#.

На цьому етапі форма покупця ще не потрібна: спочатку зрозуміємо переходи й збереження. Сервіс викликається безпосередньо, оскільки C# у Interactive Server уже працює на сервері. HTTP-запит до власного API не є обов'язковим.

## 2. Модель збереженого замовлення

Створюємо нові типи, а не перетворюємо `OrderState` на таблицю. `PurchaseOrder` — заголовок замовлення, `PurchaseItem` — його рядок. Зв'язок «одне замовлення — багато позицій» забезпечує зовнішній ключ `PurchaseOrderId`.

`OrderLine` залишається підтвердженою позицією тимчасового кошика. `PurchaseItem` зберігає знімок назви, опцій і ціни: перейменування товару завтра не переписує вчорашнє замовлення.

`[NotMapped]` означає, що обчислювана сума не має окремого стовпця. Поля покупця поки порожні; їх відразу включено до схеми для форми в наступній роботі. `RequestId` — ключ однієї спроби оформлення, не номер покупця і не засіб авторизації.

**Адаптуйте зараз:** формат опису опцій для власної теми. Універсальні назви `ProductName`, `Quantity` і `UnitPrice` можна залишити.

### Файл `Models/PurchaseOrder.cs`

**Що описує код.** У файлі два класи, бо замовлення має загальні дані та довільну кількість позицій. Наприклад, одне замовлення містить два капучино й один чай: це один `PurchaseOrder` і два об'єкти `PurchaseItem`, а не три замовлення.

- `PurchaseOrder.Id` — номер запису, який після збереження можна використати в URL. `PurchaseItem.Id` — номер окремого рядка.
- `PurchaseOrderId` указує, якому замовленню належить рядок. Колекція `Items` дозволяє працювати з цими рядками як з частиною об'єкта замовлення.
- `RequestId` ідентифікує операцію оформлення. Він потрібен для розпізнавання повторної спроби, а `Id` — для пошуку вже збереженого замовлення.
- `CreatedUtc` фіксує момент створення в UTC. Тому в розмітці далі явно показано позначку UTC.
- `UnitPrice` — ціна однієї одиниці з обраними опціями. `Total` рядка множить її на `Quantity`, а `Total` замовлення додає суми всіх рядків.

Запис `=>` у властивості `Total` означає обчислення при зверненні. Для двох одиниць по 100 грн результат буде 200 грн. `[NotMapped]` повідомляє EF Core, що окремий стовпець для цього результату не потрібний. Тип `decimal` використано для грошових значень.

**Перевірте розуміння:** що станеться з історією, якщо замість знімка `ProductName` щоразу читати поточну назву з каталогу? Які дані повинні зберігати ваші власні позиції, щоб старе замовлення залишалося зрозумілим після зміни каталогу?

Створіть файл моделі за наведеним лістингом, узгодивши назви та опис опцій зі своєю темою:

```
using System.ComponentModel.DataAnnotations.Schema;

namespace CampusCafe.Models;

public class PurchaseOrder
{
    public int Id { get; set; }
    public Guid RequestId { get; set; }
    public DateTime CreatedUtc { get; set; }
    // У лабораторній 9 ці поля порожні; форма з'явиться в лабораторній 10.
    public string CustomerName { get; set; } = "";
    public string Phone { get; set; } = "";
    public string DeliveryAddress { get; set; } = "";
    public string Comment { get; set; } = "";
    public List<PurchaseItem> Items { get; set; } = new();
    [NotMapped] public decimal Total => Items.Sum(item => item.Total);
}

public class PurchaseItem
{
    public int Id { get; set; }
    public int PurchaseOrderId { get; set; }
    public string ProductName { get; set; } = "";
    public string Options { get; set; } = "";
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
```

```
    [NotMapped] public decimal Total => UnitPrice * Quantity;
}
```

## 3. База замовлень без втрати каталогу

Каталог попередньої роботи залишається у `cafe.db`. Для замовлень створимо окремий `orders.db` і відповідний `OrderDbContext`. Це свідоме навчальне розділення: не видаляємо каталог і не змішуємо оновлення його схеми з вивченням маршрутизації.

`EnsureCreated` створює відсутню схему, але не додає нові таблиці до вже створеної схеми каталогу й не замінює міграції. Саме тому не достатньо просто дописати `DbSet` до старого контексту. У цих трьох роботах схема замовлень після першого створення залишається сталою; для подальшої зміни моделі потрібен окремий план міграцій.

Індекс `RequestId` унікальний: база не дозволить двом однаковим спробам створити два записи. Один `SaveChangesAsync` зберігає заголовок і його рядки як одну операцію. Між двома файлами БД немає спільної транзакції; конкурентне редагування цін адміністратором не входить у цю демонстрацію.

### Файл `Data/OrderDbContext.cs`

**Роль контексту.** `DbContext` — об'єкт для виконання операцій EF Core з базою. Він отримує налаштування підключення через `DbContextOptions<OrderDbContext>`. Сам шлях до SQLite буде задано в `Program.cs`, тому модель і контекст не містять жорстко записаної адреси файла.

`DbSet<PurchaseOrder> Orders` є точкою входу для запитів і додавання замовлень. Через зв'язок `Items` EF Core також включає `PurchaseItem` до моделі, хоча окрема властивість `DbSet<PurchaseItem>` тут не оголошена.

У `OnModelCreating` прочитайте ланцюжок налаштувань словами: «одне замовлення має багато позицій; позиція належить одному замовленню через `PurchaseOrderId`; видалення замовлення видаляє його позиції». `WithOne()` без аргументу означає, що в `PurchaseItem` немає окремої навігаційної властивості з посиланням на об'єкт замовлення.

`HasIndex(...).IsUnique()` забезпечує обмеження в самій БД. Попередньої перевірки в C# недостатньо: два запити можуть одночасно не знайти запис, а потім обидва спробувати його створити.

**Перевірте розуміння:** чому зовнішній ключ і унікальний індекс виконують різні завдання? Перший пов'язує позицію із замовленням, другий забороняє повторення ключа операції.

Створіть контекст із цими правилами:

```
using CampusCafe.Models;
using Microsoft.EntityFrameworkCore;

namespace CampusCafe.Data;

public class OrderDbContext(DbContextOptions<OrderDbContext> options) : DbContext(options)
```

```
{
    public DbSet<PurchaseOrder> Orders => Set<PurchaseOrder>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<PurchaseOrder>().HasIndex(o => o.RequestId).IsUnique();
        modelBuilder.Entity<PurchaseOrder>().HasMany(o => o.Items).WithOne()
            .HasForeignKey(i => i.PurchaseOrderId).OnDelete(DeleteBehavior.Cascade);
    }
}
```

## 4. Сервіс перевірки та збереження

`PurchaseService` відділяє сторінки від запитів до SQLite. `PlaceAsync` перевіряє кошик, повторно знаходить товари й обчислює ціну за серверними правилами. Якщо ціна змінилася, замовлення відхиляється з поясненням, а не зберігається за іншою сумою без згоди користувача.

Не довіряємо назві й ціні, переданим разом із позицією. Беремо актуальну назву з каталогу. `ToArray()` у сторінці зафіксує набір позицій на момент натискання. Ліміт 50 стосується рядків, а правило 1–10 — кількості одиниць у рядку.

`GetAllAsync` повертає список, `GetAsync` — один запис або `null`. `Include` завантажує рядки разом із заголовком. Кожен метод створює й звільняє власний контекст через фабрику.

Параметри в дужках після `class PurchaseService` — основний конструктор C#. DI передає в нього фабрику й сервіс каталогу; це коротший запис звичайного конструктора, а не виклик методу. Таку саму форму використано для `OrderDbContext`.

**Адаптуйте зараз:** відновлення чернетки з позиції, допустимі опції та текст `Options`. Для букета тут мають бути квіти й пакування, а не молоко. Не вилучайте повторну перевірку ціни.

### Файл `Services/PurchaseService.cs`

**Як читати найбільший метод.** `PlaceAsync` має послідовні етапи. Знайдіть кожен із них у лістингу, перш ніж переносити код:

1. Перевірка `requestId`: порожній `Guid` не може позначати конкретну спробу оформлення.
2. Пошук попереднього результату за тим самим ключем. Якщо запис уже існує, повертається його номер без створення дубля.
3. Перевірка кількості рядків і завантаження каталогу. `ToDictionary(p => p.Id)` дозволяє шукати товар за ідентифікатором.
4. Для кожного рядка відновлюється `DrinkDraft`. Метод `CreateLine` застосовує вже відомі правила допустимих опцій, кількості та розрахунку ціни. Отже, одна й та сама логіка використовується при додаванні до кошика і при оформленні.
5. Порівнюється заново обчислена ціна з тією, яку підтверджував користувач. Сервіс не підмінює суму мовчки.
6. Створюються знімки `PurchaseItem`, після чого `Add` позначає замовлення як нове для EF Core. Фактичний запис виконує `SaveChangesAsync`, а не сам `Add`.

7. Після успіху повертається `order.Id`, який використає сторінка для переходу.

`await using` означає, що контекст буде асинхронно звільнений після завершення його області, зокрема при винятку. Фабрика дає кожній операції власний короткоживучий контекст, замість спільного контексту на весь час роботи сторінок.

**Навіщо блок `catch`.** Якщо паралельна спроба вже записала те саме замовлення, унікальний індекс може відхилити другий запис. Код відкриває новий контекст і шукає результат за `RequestId`. Якщо результат знайдено, повертає його номер. Якщо ні — `throw;` передає помилку вище, а не приховує будь-яку несправність БД як успіх.

**Як працює читання.** `AsNoTracking()` вимикає відстеження змін для перегляду. `Include(o => o.Items)` завантажує позиції, потрібні для підсумку. Без такого завантаження не можна покладатися на правильність суми за порожньою початковою колекцією. `SingleOrDefaultAsync` повертає один об'єкт або `null`, якщо його немає. `Task<int>` означає асинхронну операцію, результатом якої буде ціле число.

**Мініперевірка:** для своєї позиції вручну обчисліть ціну та знайдіть рядок коду, який перевірить її на сервері. Окремо поясніть, чому два однакові товари у двох різних замовленнях не вважаються дублікатом: повторення визначає ключ операції, а не схожість складу.

Створіть сервіс, зберігши цю послідовність перевірок:

```
using CampusCafe.Data;
using CampusCafe.Models;
using Microsoft.EntityFrameworkCore;

namespace CampusCafe.Services;

public class PurchaseService(
    IDbContextFactory<OrderDbContext> factory,
    CatalogService catalog)
{
    public async Task<int> PlaceAsync(IReadOnlyList<OrderLine> lines, Guid requestId)
    {
        if (requestId == Guid.Empty)
            throw new ArgumentException("Відсутній ідентифікатор операції.");
        await using var db = await factory.CreateDbContextAsync();
        var existing = await db.Orders.SingleOrDefaultAsync(o => o.RequestId == requestId);
        if (existing is not null) return existing.Id;
        if (lines.Count is 0 or > 50)
            throw new ArgumentException("Замовлення має містити від 1 до 50 позицій.");

        var products = (await catalog.GetDrinksAsync()).ToDictionary(p => p.Id);
        var order = new PurchaseOrder { RequestId = requestId, CreatedUtc = DateTime.UtcNow
};
        foreach (var line in lines)
        {
            if (!products.TryGetValue(line.DrinkId, out var product))
                throw new ArgumentException("Товар більше недоступний.");
            var draft = new DrinkDraft
            {
                Volume = line.Volume,
```

```
                Quantity = line.Quantity,
                Milk = line.Milk switch
                {
                    "звичайне" => MilkKind.Regular,
                    "вівсяне" => MilkKind.Oat,
                    _ => throw new ArgumentException("Невідомий вид молока.")
                }
            };
            var checkedLine = draft.CreateLine(product);
            // Не приймаємо підмінену або застарілу ціну без згоди користувача.
            if (checkedLine.UnitPrice != line.UnitPrice)
                throw new ArgumentException("Ціна змінилася. Видаліть позицію й додайте її
з каталогу знову.");
            order.Items.Add(new PurchaseItem
            {
                ProductName = product.Name,
                Options = $"{checkedLine.Volume} мл · {checkedLine.Milk} молоко",
                Quantity = checkedLine.Quantity,
                UnitPrice = checkedLine.UnitPrice
            });
        }
        db.Orders.Add(order);
        try { await db.SaveChangesAsync(); }
        catch (DbUpdateException)
        {
            // Повтор одночасного запиту з тим самим ключем не створює дубль.
            await using var retry = await factory.CreateDbContextAsync();
            var saved = await retry.Orders.AsNoTracking()
                .SingleOrDefaultAsync(o => o.RequestId == requestId);
            if (saved is not null) return saved.Id;
            throw;
        }
        return order.Id;
    }

    public async Task<List<PurchaseOrder>> GetAllAsync()
    {
        await using var db = await factory.CreateDbContextAsync();
        return await db.Orders.AsNoTracking().Include(o => o.Items)
            .OrderByDescending(o => o.Id).ToListAsync();
    }

    public async Task<PurchaseOrder?> GetAsync(int id)
    {
        await using var db = await factory.CreateDbContextAsync();
        return await db.Orders.AsNoTracking().Include(o => o.Items)
            .SingleOrDefaultAsync(o => o.Id == id);
    }
}
```

## 5. Реєстрація сервісів і підготовка БД

Нових NuGet-пакетів не потрібно: SQLite уже підключений. Реєстрації виконуються до `builder.Build()`, а створення відсутньої БД — до `app.Run()`.

Каталог і замовлення мають різні типи контекстів, тому DI може надати потрібну фабрику. `PurchaseService` отримує залежності через конструктор. Створення класу в папці `Services` саме по собі не реєструє його.

### Файл `Program.cs`

**Що додається до запуску.** `Path.Combine` формує шлях від кореня застосунку до `orders.db`. `UseSqlite` налаштовує провайдер і рядок підключення. `AddDbContextFactory<OrderDbContext>` навчає DI створювати фабрику саме цього контексту, а `AddScoped<PurchaseService>` дозволяє сторінкам отримувати сервіс через `@inject`.

Після `Build` контейнер уже зібраний. `GetRequiredService` отримує зареєстровану фабрику, створюється контекст і виконується `EnsureCreatedAsync`. Це підготовка сховища, а не створення тестового замовлення. Якщо схема вже існує, цей виклик не перебудовує її за новою моделлю.

**Контрольна точка:** після додавання моделі, контексту, сервісу та реєстрацій зберіть і запустіть застосунок. Перевірте появу `orders.db` та доступність попереднього каталогу. Наявність файла ще не доводить, що замовлення збережене: це перевіримо після створення сторінки.

Внесіть локальні зміни, залишивши інші налаштування запуску:

```
--- a/Program.cs
+++ b/Program.cs
@@ -13,6 +13,11 @@
     options.UseSqlite($"Data Source={databasePath}"));
 builder.Services.AddScoped<CatalogService>();
 builder.Services.AddScoped<OrderState>();
+// Каталог попередньої роботи не змінюємо. Замовлення мають окрему навчальну БД.
+var ordersPath = Path.Combine(builder.Environment.ContentRootPath, "orders.db");
+builder.Services.AddDbContextFactory<OrderDbContext>(options =>
+    options.UseSqlite($"Data Source={ordersPath}"));
+builder.Services.AddScoped<PurchaseService>();

 var app = builder.Build();

@@ -31,5 +36,10 @@

 await SeedData.InitializeAsync(
     app.Services.GetRequiredService<IDbContextFactory<CafeDbContext>>());
+await using (var ordersDb = await app.Services
+    .GetRequiredService<IDbContextFactory<OrderDbContext>>().CreateDbContextAsync())
+{
+    await ordersDb.Database.EnsureCreatedAsync();
+}

 app.Run();
```

## 6. Сторінка перевірки кошика

Спільний компонент `OrderReview` лише показує позиції. Він не зберігає їх і не очищує кошик. Сторінка `Checkout` координує дію: виклик сервісу, очищення й перехід на нову адресу.

`isSubmitting` блокує повторне натискання під час очікування; `finally` повертає кнопку в робочий стан навіть після помилки. `requestId` не змінюється при повторній спробі в цьому екземплярі сторінки. Тому якщо результат збереження був втрачений, повтор не створить дубль. Це не глобальна заборона двох схожих замовлень із різних вкладок.

У звичайному HTML-елементі пишемо `disabled="@isSubmitting"`: знак `@` означає значення C#-змінної. Запис `disabled="isSubmitting"` був би буквальним HTML-атрибутом і постійно блокував би кнопку, незалежно від значення поля.

Спочатку створіть обидва файли, потім перевірте збирання. Прямий перехід на `/checkout` із порожнім кошиком повинен показувати пояснення, а не виняток.

## Файл `Components/Shared/OrderReview.razor`

**Розбір відображення.** Вхід `Items` позначено `[Parameter]`: список надає батьківська сторінка. `IReadOnlyList` показує, що цей компонент має читати колекцію, а не додавати або видаляти її елементи. `[EditorRequired]` нагадує автору розмітки передати параметр, але не виконує бізнес-перевірку замовлення.

`foreach` створює рядок інтерфейсу для кожної позиції. `@key="item.Id"` пов’язує елемент розмітки з конкретною позицією при оновленні списку. `MoneyLabel` повторно використовує оформлення суми з попередньої роботи. Підсумок обчислюється зі значень позицій, а не з тексту в браузері.

**Адаптація:** змініть лише опис товару й опцій. Не додавайте сюди запити до SQLite: цей компонент надалі використаємо і всередині форми.

Створіть компонент перегляду:

```
<ul class="review-lines">
    @foreach (var item in Items)
    {
        <li @key="item.Id">
            <strong>@item.DrinkName</strong>
            <span>@item.Volume мл · @item.Milk молоко · @item.Quantity шт.</span>
            <MoneyLabel Amount="item.Total" />
        </li>
    }
</ul>
<p class="review-total">Разом: <MoneyLabel Amount="Items.Sum(i => i.Total)" /></p>

@code {
    [Parameter, EditorRequired] public IReadOnlyList<OrderLine> Items { get; set; } =
Array.Empty<OrderLine>();
}
```

## Файл `Components/Pages/Checkout.razor`

**Розбір сторінки.** `@page` задає URL, а чотири `@inject` надають різні інструменти: `Cart` читає кошик, `Purchases` зберігає, `Navigation` змінює адресу, `Logger` записує технічні подробиці помилок. Жоден `@inject` сам по собі не запускає збереження.

У розмітці `if` відокремлює порожній кошик від доступного оформлення. У методі спочатку спрацьовує захист від повторного натискання, потім очищується попереднє повідомлення. `ToArray()` створює окремий масив поточних позицій, але не є універсальним глибоким копіюванням будь-яких об’єктів.

Зверніть увагу на порядок: `await PlaceAsync` → `Cart.Clear()` → `NavigateTo`. Якщо сервіс завершується винятком, наступні два рядки не виконуються. Саме тому при відмові збереження користувач не втрачає кошик. `finally` виконується і при успіху, і при помилці.

`requestId` створюється один раз для екземпляра сторінки. Якби `Guid.NewGuid()` стояв усередині кожної спроби `PlaceOrderAsync`, повтор уже не розпізнавався б як та сама операція. Повне перезавантаження створює новий екземпляр і новий ключ; відновлення ключа після перезавантаження цей приклад не реалізує.

**Перевірте прогноз:** який текст побачить користувач і чи очиститься кошик, якщо товар зник із каталогу? Знайдіть у сервісі місце появи винятку, а тут — місце його обробки.

Створіть сторінку та перевірте обидві гілки інтерфейсу:

```
@page "/checkout"
@inject OrderState Cart
@inject PurchaseService Purchases
@inject NavigationManager Navigation
@inject ILogger<Checkout> Logger

<PageTitle>Перевірка замовлення</PageTitle>
<h1>Перевірка замовлення</h1>
<p>Навчальне замовлення без оплати. Форму покупця додамо в наступній роботі.</p>
@if (Cart.Items.Count == 0)
{
    <p>Кошик порожній. <a href="/">Повернутися до каталогу</a></p>
}
else
{
    <div class="panel">
        <OrderReview Items="Cart.Items" />
        @if (error is not null) { <p role="alert" class="validation-message">@error</p> }
        <button type="button" disabled="@isSubmitting" @onclick="PlaceOrderAsync">
            @(isSubmitting ? "Збереження…" : "Зберегти замовлення")
        </button>
        <a class="back-link" href="/">Повернутися до каталогу</a>
    </div>
}

@code {
    private bool isSubmitting;
    private string? error;
    private readonly Guid requestId = Guid.NewGuid();

    private async Task PlaceOrderAsync()
    {
        if (isSubmitting) return;
        isSubmitting = true;
        error = null;
```

```
        try
        {
            var id = await Purchases.PlaceAsync(Cart.Items.ToArray(), requestId);
            Cart.Clear();
            Navigation.NavigateTo($"/orders/{id}");
        }
        catch (ArgumentException ex) { error = ex.Message; }
        catch (InvalidOperationException ex) { error = ex.Message; }
        catch (Exception ex)
        {
            Logger.LogError(ex, "Помилка збереження замовлення");
            error = "Не вдалося зберегти замовлення. Кошик не очищено. Спробуйте ще раз.";
        }
        finally { isSubmitting = false; }
    }
}
```

## 7. Список і деталі замовлення

`/orders` відображає всі тестові замовлення. Не називаємо цей список «Мої замовлення»: немає входу й перевірки власника.

`/orders/{Id:int}` приймає ціле число. `/orders/abc` не відповідає маршруту; `/orders/999999` може відповідати маршруту, але запису в БД немає. Це різні ситуації. Помилка доступу до БД теж не дорівнює «запис не знайдено».

Для деталей використовуємо `OnParametersSetAsync`, а не тільки `OnInitializedAsync`: маршрутизатор може повторно використати той самий компонент із новим `Id`. Лічильник `loadVersion` не дозволяє запізнілому результату старого завантаження замінити новий.

**Контроль:** після збереження відкрийте номер замовлення в окремій вкладці. Кошик там порожній, але запис має бути доступний із БД.

### Файл `Components/Pages/Orders.razor`

**Чотири стани списку.** `orders == null` означає, що дані ще завантажуються. Порожній список означає успішний запит без записів. Непорожній список відображається картками. `error` означає невдалий запит і має пріоритет у розмітці.

`OnInitializedAsync` запускає перше читання. Кнопка «Повторити» викликає той самий `LoadAsync`, тому не потрібно дублювати логіку. Перед спробою метод скидає старі дані й помилку. Посилання в картці утворюється з `order.Id`, а кількість одиниць визначає `Sum(i => i.Quantity)`, не кількість рядків `Items.Count`.

**Контроль:** створіть замовлення з двома рядками по кілька одиниць і порівняйте сумарну кількість із кількістю рядків. Поясніть, чому повідомлення «замовлень немає» не можна показувати при помилці з’єднання з БД.

Створіть сторінку списку:

```
@page "/orders"
@inject PurchaseService Purchases
@inject ILogger<Orders> Logger

<PageTitle>Навчальні замовлення</PageTitle>
<h1>Навчальні замовлення</h1>
<p>Це спільний демонстраційний список без облікових записів. Не вводьте реальні персональні
дані.</p>
@if (error is not null)
{
    <p role="alert">@error</p>
    <button @onclick="LoadAsync">Повторити</button>
}
else if (orders is null) { <p role="status">Завантаження…</p> }
else if (orders.Count == 0) { <p>Замовлень ще немає. <a href="/">Відкрити каталог</a></p> }
else
{
    <div class="orders-list">
        @foreach (var order in orders)
        {
            <article class="panel" @key="order.Id">
                <h2><a href="@($"/orders/{order.Id}")">Замовлення №@order.Id</a></h2>
                <p>@order.CreatedUtc.ToString("dd.MM.yyyy HH:mm") UTC · @order.Items.Sum(i
=> i.Quantity) шт.</p>
                <MoneyLabel Amount="order.Total" />
            </article>
        }
    </div>
}

@code {
    private List<PurchaseOrder>? orders;
    private string? error;
    protected override Task OnInitializedAsync() => LoadAsync();
    private async Task LoadAsync()
    {
        error = null;
        orders = null;
        try { orders = await Purchases.GetAllAsync(); }
        catch (Exception ex)
        {
            Logger.LogError(ex, "Не вдалося отримати замовлення");
            error = "Не вдалося отримати список. Перевірте сервер.";
        }
    }
}
```

## Файл `Components/Pages/OrderDetails.razor`

**Як URL стає запитом.** Для `/orders/12` маршрутизатор записує `12` у `[Parameter] public int Id`. Після встановлення параметрів викликається `OnParametersSetAsync`, який передає номер у `Purchases.GetAsync(Id)`. Суфікс `:int` перевіряє тип фрагмента URL, але не існування запису.

**Захист від застарілого результату.** При першому переході `version` може дорівнювати 1. Якщо до завершення запиту користувач відкриє інший номер, `loadVersion` стане 2. Результат першого запиту і його `finally` не змінять новий екран, оскільки `1 != 2`. Це ігнорування застарілого результату, а не скасування SQL-запиту.

Окремий `loading` потрібний тому, що `order == null` може означати як очікування, так і успішну відповідь «не знайдено». Розмітка перевіряє помилку, очікування, відсутність і лише потім дані.

**Контроль:** введіть вручну адресу існуючого та неіснуючого номера, потім `/orders/abc`. Для кожного випадку назвіть, де закінчилась обробка: у маршрутизаторі чи в запиті до БД.

Створіть сторінку деталей:

```
@page "/orders/{Id:int}"
@inject PurchaseService Purchases
@inject ILogger<OrderDetails> Logger

<PageTitle>Деталі замовлення</PageTitle>
<h1>Замовлення №@Id</h1>
@if (error is not null) { <p role="alert">@error</p> }
else if (loading) { <p role="status">Завантаження…</p> }
else if (order is null) { <p>Такого замовлення немає.</p> }
else
{
    <article class="panel">
        <p>Створено: @order.CreatedUtc.ToString("dd.MM.yyyy HH:mm") UTC</p>
        <ul class="review-lines">
            @foreach (var item in order.Items)
            {
                <li @key="item.Id">
                    <strong>@item.ProductName</strong>
                    <span>@item.Options · @item.Quantity шт.</span>
                    <MoneyLabel Amount="item.Total" />
                </li>
            }
        </ul>
        <p class="review-total">Разом: <MoneyLabel Amount="order.Total" /></p>
        <p>Замовлення збережено. Оплата не проводилася.</p>
    </article>
}
<a class="back-link" href="/orders">До списку замовлень</a>

@code {
    [Parameter] public int Id { get; set; }
    private PurchaseOrder? order;
    private bool loading = true;
    private string? error;
    private int loadVersion;

    protected override async Task OnParametersSetAsync()
    {
        var version = ++loadVersion;
        loading = true;
        error = null;
        order = null;
```

```
        try
        {
            var result = await Purchases.GetAsync(Id);
            if (version == loadVersion) order = result;
        }
        catch (Exception ex)
        {
            Logger.LogError(ex, "Не вдалося отримати замовлення {Id}", Id);
            if (version == loadVersion) error = "Помилка читання даних. Це не означає, що
замовлення відсутнє.";
        }
        finally { if (version == loadVersion) loading = false; }
    }
}
```

## 8. Єдина навігація й оформлення

Не копіюйте меню в кожну сторінку. Доповніть наявний `MainLayout`: стандартне зіставлення префікса підсвітить «Замовлення» також на `/orders/12`. Для головної сторінки вже задано `NavLinkMatch.All`.

У кошику посилання `/checkout` з'являється тільки за наявності позицій. Окремий `orders.css` доповнює ваші стилі, не переписуючи оформлення каталогу.

### Файл `Components/Layout/MainLayout.razor`

`MainLayout` обгортає сторінки спільною шапкою й меню. Новий `NavLink` не читає БД: він лише задає адресу та підсвічування. Завдяки зіставленню префікса секція «Замовлення» залишиться активною і на сторінці окремого запису. `Match="NavLinkMatch.All"` для `/` запобігає підсвічуванню головної сторінки для кожної адреси.

Після правки відкрийте список і деталі: меню повинно залишитися однаковим, а вміст сторінки змінитися. Внесіть локальну зміну:

```
--- a/Components/Layout/MainLayout.razor
+++ b/Components/Layout/MainLayout.razor
@@ -5,6 +5,7 @@
         <a class="brand" href="/" aria-label="Campus Café — головна"><span aria-
hidden="true">☕</span> @settings.Name</a>
         <nav aria-label="Основна навігація">
             <NavLink href="/" Match="NavLinkMatch.All">Меню</NavLink>
+            <NavLink href="/orders">Замовлення</NavLink>
             <NavLink href="/about">Про кав'ярню</NavLink>
         </nav>
     </header>
```

### Файл `Components/Shared/CartSummary.razor`

Посилання додається в гілку непорожнього кошика. Натискання лише відкриває перевірку, а не зберігає замовлення. Це розділяє навігацію та підтвердження дії. Пряме відкриття `/checkout` все

одно перевіряється самою сторінкою: приховане посилання не є захистом.

Внесіть локальні зміни:

```
--- a/Components/Shared/CartSummary.razor
+++ b/Components/Shared/CartSummary.razor
@@ -28,8 +28,9 @@
             <span>До сплати</span><strong><MoneyLabel Amount="Order.Total" /></strong>
         </div>
         <button type="button" class="secondary full-width" @onclick="Order.Clear">Очистити
кошик</button>
+        <a class="button-link checkout-link" href="/checkout">Перевірити замовлення</a>
     }
-    <p class="cart-note">Навчальний кошик. Оплата та збереження замовлень не виконуються.
</p>
+    <p class="cart-note">Кошик тимчасовий. Замовлення зберігається після підтвердження на
окремій сторінці. Оплати немає.</p>
 </aside>

 @code {
```

## Файл `wwwroot/orders.css`

Стилі відповідають за читабельність уже наявної розмітки. `.panel` оформлює блок, `.review-lines li` розміщує частини позиції через Flexbox, а `flex-wrap` дозволяє переносити їх на вузькому екрані. `button:disabled` лише показує стан кнопки; справжнє блокування задає атрибут `disabled` у Razor. CSS не виконує перевірку даних і не запобігає повторному запису до БД.

Створіть файл стилів і збережіть власні кольори. Перевірте довгу назву товару на вузькому екрані:

```
.panel { background:#fff; border-radius:8px; padding:24px; margin:20px 0; box-shadow:0 2px
7px #0002; }
.review-lines { list-style:none; padding:0; }
.review-lines li { display:flex; flex-wrap:wrap; justify-content:space-between; gap:12px;
padding:14px 0; border-bottom:1px solid #ddd; }
.review-lines li span { color:#555; }
.review-total { font-size:1.3rem; font-weight:700; }
.orders-list { max-width:850px; }
.checkout-link { display:block; margin-top:14px; text-align:center; }
.back-link { display:inline-block; margin:18px; }
button:disabled { opacity:.55; cursor:not-allowed; }
nav { flex-wrap:wrap; }
```

## Файл `Components/App.razor`

Створений CSS ще потрібно підключити до HTML-документа. Новий `<link>` додається поряд із наявними стилями, а `Assets` визначає URL ресурсу. Налаштування `Routes`, `HeadOutlet` та інтерактивності не змінюються.

**Контроль:** якщо функції працюють, але блоки не оформлені, перевірте саме ім’я файла, його розташування у `wwwroot` та цей `<link>`, а не сервіс замовлень.

Внесіть локальну зміну:

```
--- a/Components/App.razor
+++ b/Components/App.razor
@@ -7,6 +7,7 @@
     <base href="/" />
     <ResourcePreloader />
     <link rel="stylesheet" href="@Assets["app.css"]" />
+    <link rel="stylesheet" href="@Assets["orders.css"]" />
     <link rel="stylesheet" href="@Assets["CampusCafe.styles.css"]" />
     <ImportMap />
     <HeadOutlet @rendermode="new InteractiveServerRenderMode(prerender: false)" />
```

## 9. Контрольні сценарії та самостійна частина

| Дія | Очікуваний результат |
|---|---|
| Відкрити `/checkout` із порожнім кошиком | Посилання повернення до каталогу |
| Додати капучино 350 мл, вівсяне, 2 шт. | Підсумок 200 грн |
| Зберегти | Нова адреса `/orders/номер`, кошик очищено |
| Оновити сторінку та перезапустити сервер | Збережене замовлення доступне |
| Відкрити `/orders/abc` | Сторінка невідомого маршруту |
| Відкрити неіснуючий числовий номер | Повідомлення про відсутній запис |
| Переглянути список | Посилання ведуть до правильних номерів |
| Повторити той самий ключ через автоматичну перевірку | Один запис, а не два |

**Самостійно:** адаптуйте підсумок і деталі до власних товарів; змініть тексти переходів; наведіть два різні замовлення з ручною перевіркою суми. Підготуйте карту власних маршрутів і поясніть, яка сторінка отримує параметр.

Для порівняння рядка маршруту й query-параметра необов’язково додайте до списку фільтр за номером через `[SupplyParameterFromQuery]`. Фільтр змінює вибір даних, а не шаблон `/orders/{Id:int}`.

## Структура результату

```
CampusCafe/
  Components/App.razor
  Components/Layout/MainLayout.razor
  Components/Layout/MainLayout.razor.css
  Components/Layout/ReconnectModal.razor
  Components/Layout/ReconnectModal.razor.css
  Components/Layout/ReconnectModal.razor.js
  Components/Pages/About.razor
```

```
Components/Pages/Checkout.razor
Components/Pages/Error.razor
Components/Pages/Home.razor
Components/Pages/NotFound.razor
Components/Pages/OrderDetails.razor
Components/Pages/Orders.razor
Components/Routes.razor
Components/Shared/CartSummary.razor
Components/Shared/ConfigureDrinkDialog.razor
Components/Shared/DrinkCard.razor
Components/Shared/MoneyLabel.razor
Components/Shared/OrderReview.razor
Components/_Imports.razor
Data/CafeDbContext.cs
Data/OrderDbContext.cs
Data/SeedData.cs
Models/Drink.cs
Models/DrinkDraft.cs
Models/OrderLine.cs
Models/PurchaseOrder.cs
Models/ShopSettings.cs
Program.cs
Services/CatalogService.cs
Services/OrderState.cs
Services/PurchaseService.cs
wwwroot/app.css
wwwroot/orders.css
```

## Підсумок і запитання для захисту

`@page` визначає адресу, `[Parameter]` отримує номер, `OnParametersSetAsync` реагує на зміну номера. `MainLayout` прибирає дублювання меню. `PurchaseService` перевіряє й зберігає, а сторінка координує взаємодію. Кошик і збережене замовлення — різні об'єкти з різним часом життя.

Нові файли: `Models/PurchaseOrder.cs` — заголовок і рядки; `Data/OrderDbContext.cs` — схема замовлень; `Services/PurchaseService.cs` — операції; `Shared/OrderReview.razor` — підсумок; сторінки `Checkout`, `Orders`, `OrderDetails`; `wwwroot/orders.css` — додаткові стилі.

Змінені файли: `Program.cs`, `App.razor`, `MainLayout.razor`, `CartSummary.razor`. Новий `orders.db` створюється під час запуску, не копіюється як вихідний код. `cafe.db` залишається на місці.

Запитання: чому ідентифікатор у URL не є дозволом на доступ? Чим відрізняються помилка БД і відсутній запис? Чому `Cart.Clear()` стоїть після `await PlaceAsync`? Навіщо зберігати знімок назви й ціни?

## Запуск і автоматичні перевірки

Із кореня готового комплекту:

```
dotnet run --project CampusCafe --urls http://localhost:5127
```

Для власного проєкту з його папки достатньо `dotnet run --urls http://localhost:5127`. Не запускайте кілька етапів на одному порту. Зупинка — `Ctrl+C`.

Окремо запустіть перевірки демо:

```
dotnet run --project CampusCafe.Checks
```

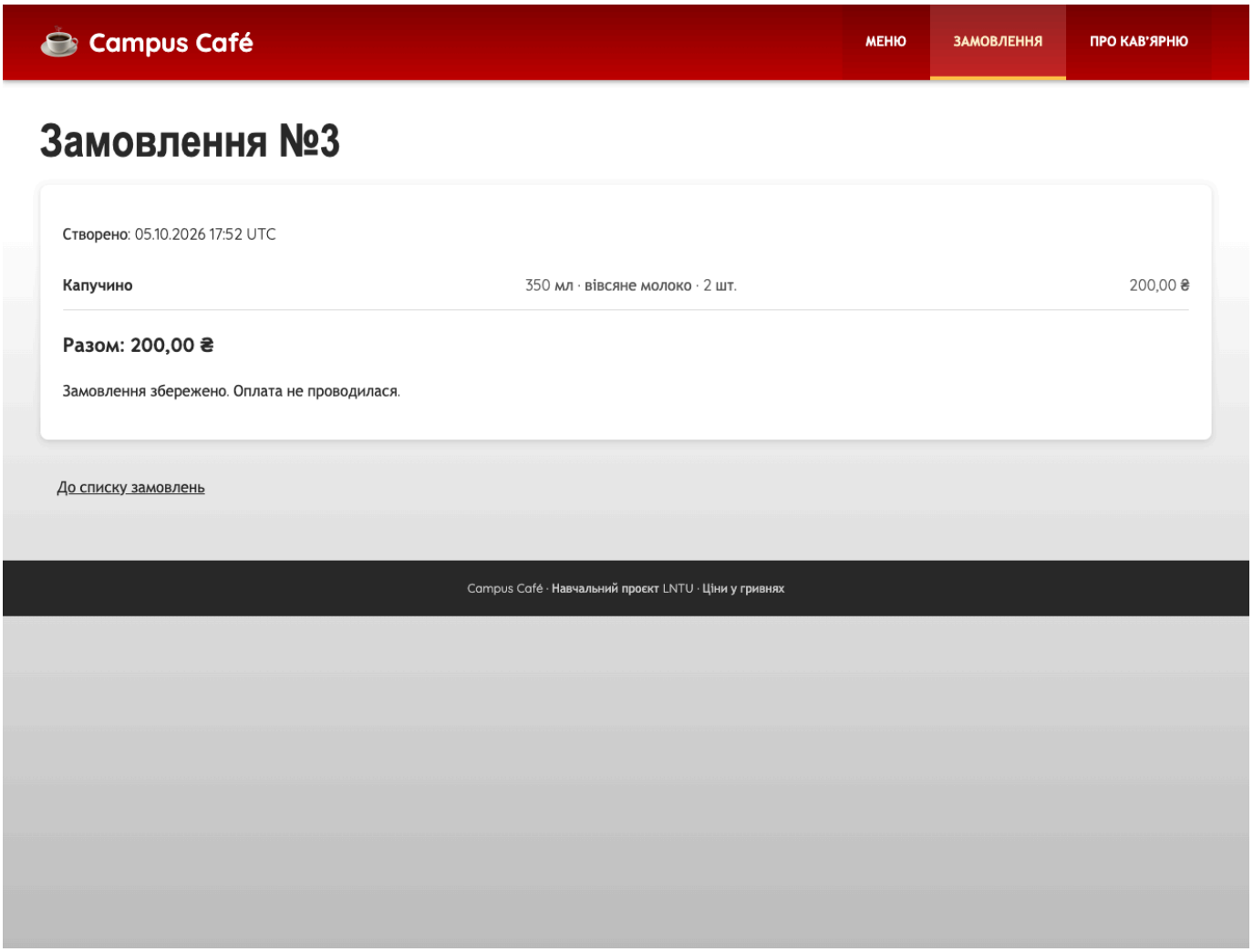
Тести працюють з окремими SQLite-базами в пам'яті й не змінюють робочі файли. Це перевірки C# і сховища, а не заміна ручного тестування браузера. Проєкт перевірок можна взяти з комплекту поточного етапу; для власних моделей адаптуйте типи, назви й контрольні ціни. Не запускайте старі тести зі старими сигнатурами сервісів.

Для здавання підготуйте вихідний код без `bin`, `obj` і персональних даних, короткий README, результати контрольних сценаріїв та пояснення власної тематичної адаптації. Збережіть окремий працездатний знімок цього етапу.

Наступний етап — лабораторна 10.

## Джерела для поглиблення

- Маршрутизація.
- Навігація.
- EF Core у Blazor.
- Обмеження EnsureCreated та відмінність від міграцій.




## Короткий підсумок роботи

**Що реалізовано.** До каталогу й тимчасового кошика додано перевірку замовлення, збереження до SQLite, список замовлень і перегляд окремого запису за номером. Збережені замовлення залишаються доступними після перезапуску застосунку, навіть коли кошик уже порожній.

**Як це працює:**

- **Навігація.** `@page` задає адреси сторінок. У маршруті `/orders/{Id:int}` номер надходить у параметр `Id`, а `OnParametersSetAsync` завантажує відповідний запис. `NavLink` і `NavigationManager` забезпечують переходи.
- **Постійні дані.** `PurchaseOrder` містить загальні дані замовлення, а `PurchaseItem` — знімки назви, опцій і ціни його позицій. `OrderDbContext` пов'язує ці об'єкти з окремою базою `orders.db`, не змінюючи каталог у `cafe.db`.
- **Перевірка й запис.** `PurchaseService` повторно перевіряє товари та ціни, створює замовлення і викликає `SaveChangesAsync`. Фабрика контекстів надає окремий короткоживучий контекст для кожної операції.
- **Захист від повторів.** `RequestId` та унікальний індекс дозволяють розпізнати повтор тієї самої операції. Кошик очищується лише після успішного збереження, після чого відкриваються деталі нового замовлення.

- **Зрозумілий інтерфейс.** Сторінки окремо показують завантаження, порожній результат, відсутній запис і помилку. Спільний макет містить меню, а `OrderReview` повторно використовує розмітку підсумку.

**Головне запам'ятати:** сторінка керує взаємодією, сервіс перевіряє та зберігає, контекст виконує операції з БД. Номер у URL вибирає запис, але сам по собі не надає права доступу до нього.

## Тестові питання для самоперевірки

У кожному питанні оберіть **одну правильну відповідь**. Спочатку запишіть власні відповіді, потім перевірте їх за ключами нижче. Для помилкової відповіді знайдіть відповідний фрагмент коду й поясніть його призначення.

### 1. Користувач відкрив `/orders/12`. Як сторінка деталей отримує число 12?

- А. Із назви файла `OrderDetails.razor`.
- Б. Маршрутизатор передає його у властивість `Id`, позначену `[Parameter]`.
- В. Метод `GetAllAsync` автоматично вибирає дванадцятий елемент списку.
- Г. Компонент `NavLink` записує його безпосередньо в SQLite.

### 2. Що гарантує обмеження `:int` у маршруті `/orders/{Id:int}`?

- А. Замовлення з таким номером існує.
- Б. Користувач має право переглядати замовлення.
- В. Номер є додатним.
- Г. Відповідний фрагмент адреси можна зіставити з цілим числом типу `int`.

### 3. Чому деталі завантажуються в `OnParametersSetAsync`?

- А. Той самий екземпляр компонента може отримати інший `Id` при навігації.
- Б. Цей метод автоматично повторює запит до БД щосекунди.
- В. Лише в цьому методі дозволено використовувати `await`.
- Г. Він створює таблицю замовлень перед кожним переглядом.

### 4. Чому `Cart.Clear()` виконується після `await Purchases.PlaceAsync(...)`?

- А. До `await` не можна викликати синхронні методи.
- Б. Порожній кошик потрібен сервісу для перевірки ціни.
- В. Якщо збереження завершиться помилкою, позиції повинні залишитися в кошику.
- Г. Очищення кошика автоматично фіксує транзакцію SQLite.

### 5. Для чого потрібні `RequestId` та унікальний індекс на ньому?

- А. Щоб заборонити два замовлення з однаковими товарами.
- Б. Щоб повтор однієї операції оформлення не створив ще один запис.

- В. Щоб визначити обліковий запис покупця.
- Г. Щоб номер замовлення збігався з кількістю позицій.

### 6. Навіщо в `PurchaseItem` зберігаються назва та ціна на момент оформлення?

- А. Щоб поточний каталог автоматично оновлював усі старі замовлення.
- Б. Щоб не перевіряти товар на сервері.
- В. Щоб уникнути використання зовнішнього ключа.
- Г. Щоб подальші зміни каталогу не переписували зміст історичного замовлення.

### 7. Яка роль `Include(o => o.Items)` при читанні замовлення?

- А. Завантажити позиції разом із заголовком замовлення.
- Б. Додати нові позиції до БД.
- В. Зареєструвати сервіс у DI.
- Г. Заборонити змінювати ціни в каталозі.

### 8. Що зробить `EnsureCreatedAsync`, якщо наявна схема вже створена, а ви додали нову властивість до моделі?

- А. Автоматично створить міграцію.
- Б. Видалить старі записи й перебудує таблиці.
- В. Не оновить наявну схему відповідно до зміни моделі.
- Г. Перенесе дані з `cafe.db` до `orders.db`.

### 9. Навіщо `PurchaseService` отримує фабрику контекстів?

- А. Щоб кожне читання виконувалося з браузера через JavaScript.
- Б. Щоб створювати та звільняти окремий контекст для операції.
- В. Щоб контекст зберігався в HTML-розмітці.
- Г. Щоб `SaveChangesAsync` більше не був потрібний.

### 10. Який результат відповідає успішному запиту списку без замовлень?

- А. `orders == null` під час очікування.
- Б. Заповнене повідомлення про помилку доступу до БД.
- В. Виняток у маршрутизаторі.
- Г. Непорожнє посилання на список із `Count == 0`.

### 11. Старий запит деталей завершився пізніше за новий. Що забезпечує перевірка `version == loadVersion`?

- А. Застарілий результат не замінить дані нового перегляду.
- Б. Старий SQL-запит буде гарантовано скасований на сервері БД.
- В. Новий номер замовлення буде записаний у старий рядок БД.

- Г. Браузер автоматично повернеться на попередню адресу.

### 12. Коли `NavLink` «Замовлення» без `Match="NavLinkMatch.All"` має залишатися активним?

- А. Лише коли в кошику є товар.
- Б. Лише під час завантаження SQLite.
- В. На `/orders` і на вкладеному маршруті на зразок `/orders/12`.
- Г. На будь-якій сторінці, якщо вже існує хоча б одне замовлення.

---

## Ключі відповідей і пояснення

| № | Відповідь | Чому |
|---|---|---|
| 1 | Б | Значення сегмента маршруту надходить у параметр компонента `Id`. Сервіс отримує його вже зі сторінки. |
| 2 | Г | `:int` перевіряє відповідність типу. Існування запису та право доступу потребують окремих перевірок. |
| 3 | А | Параметри можуть змінитися без створення нового екземпляра. Початкова ініціалізація сама по собі не покриває такий перехід. |
| 4 | В | Виняток із сервісу перериває звичайну послідовність, тому очищення не виконується при невдалому збереженні. |
| 5 | Б | Ключ позначає спробу оформлення, а унікальний індекс захищає від дубля, зокрема при конкурентних запитах. |
| 6 | Г | Знімок зберігає підтверджені назву, опції та ціну незалежно від майбутнього редагування каталогу. |
| 7 | А | Позиції потрібні для відображення складу й обчислення суми. Сам `Include` нічого не додає до БД. |
| 8 | В | `EnsureCreatedAsync` не замінює міграції та не перебудовує наявну схему під змінену модель. |
| 9 | Б | Кожна операція користується власним короткоживучим `DbContext`, який звільняється через `await using`. |

| № | Відповідь | Чому |
|---|---|---|
| 10 | Г | Порожній список — коректний результат читання. `null` у цій сторінці позначає очікування, а помилка має окремий стан. |
| 11 | А | Лічильник версії відсіює застаріле оновлення інтерфейсу, але не скасовує сам запит. |
| 12 | В | Зіставлення префікса зберігає активність секції при переході до деталей замовлення. |